1. What is the difference between class and object?

A class is a blueprint that defines what an object has (fields/properties) and can do (methods). An object is an actual, concrete instance of that class created in memory.

Example:

```
BankAccount account = new BankAccount();
```

Here, `BankAccount` is the class (blueprint) and `account` is the object (instance).

---

2. What is encapsulation?

Encapsulation is the practice of bundling data and methods inside a class, protecting the internal state from direct outside interference and controlling how it can be changed.

Example:

```
public class BankAccount
{
    public decimal Balance { get; private set; } // Controlled access

    public void Deposit(decimal amount) // Controlled mutation with validation
    {
        if (amount > 0) Balance += amount;
    }
}
```

---

3. Why should account balance not be public?

Because making it public allows any external code to modify it directly and bypass business rules (invariants), such as setting the balance to an invalid negative number.

Example:

```
// BAD: If Balance is public, external code can corrupt data:
account.Balance = -5000;

// GOOD: With encapsulation, mutations are forced through business logic:
account.Withdraw(5000); // Fails due to insufficient funds checks
```

---

4. What is the difference between field and property?

A field is a variable declared directly inside a class to hold raw data. A property acts as a wrapper around a field, using accessors (`get` and `set`) to control access and protect data integrity.

Example:

```
private decimal _balance; // Field: stores the raw value
```

```
public decimal Balance    // Property: wraps and protects access to _balance
{
    get => _balance;
    private set
    {
        if (value >= 0) _balance = value;
    }
}
```

---

5. Why do we use constructors?

We use constructors to initialize an object's state and ensure it has all required data as soon as it is created.

Example:

```
Customer customer = new Customer("Ahmed");
```

This prevents the creation of a customer with a null or empty name.

---

6. What is the purpose of a service class?

A service class encapsulates business rules and operations, keeping them separated from the user interface (Console/Web) to ensure the application remains clean and modular.

Example:

In our Bank Account System, the `BankService` handles execution flows such as validating credentials, calculating interest rates, and executing money transfers.

---

7. Why should we avoid huge Main methods?

A huge `Main` method violates the Single Responsibility Principle (SRP). It makes code difficult to read, test, debug, and reuse. We should break it down into specialized classes and helper methods.

---

8. What is the difference between List and Array?

An array has a fixed size defined at creation time, while a `List<T>` is dynamic and can automatically grow or shrink as items are added or removed.

Example:

```
int[] fixedArray = new int[5]; // Fixed size of 5

List<int> dynamicList = new List<int>(); // Dynamic size
dynamicList.Add(10);
```

9. When would you use Dictionary?

I use a `Dictionary<TKey, TValue>` when I need to look up values quickly using a unique key, which provides near-instantaneous $O(1)$ constant time complexity.

Example:

```
Dictionary<int, string> students = new();
students[101] = "Ahmed";
students[102] = "Sara";
```

Searching for student 101 is direct and doesn't require looping through the entire collection.

---

10. What is LINQ used for?

LINQ (Language Integrated Query) is used to query, filter, sort, group, and transform data collections in C# using a clean, readable syntax.

Example:

```
var activeMembers = members.Where(m => m.IsActive);
```

---

11. What is the difference between Where and Select?

`Where` filters the collection (returns items matching a condition). `Select` projects or transforms the collection (returns specific data or maps items to new shapes).

Example:

```
members.Where(m => m.IsActive);   // "Which items?" -> Filters active members
members.Select(m => m.FullName);  // "Which data?" -> Extracts only their names
```

---

12. What is GroupBy used for?

`GroupBy` is used to organize collection elements into groups sharing a common key. It returns a collection of `IGrouping<TKey, TElement>` objects.

Example:

```
books.GroupBy(b => b.CategoryId); // Groups books under their category IDs
```

---

13. What are Skip and Take used for?

`Skip` ignores a specified number of items in a collection, and `Take` retrieves the next specified number. They are primarily used together to implement pagination.

Example:

```
books.Skip(10).Take(10); // Skips page 1 (first 10 books) and takes page 2 (next 10)
```

---

14. What is a primary key?

A primary key is a column (or set of columns) that uniquely identifies each row in a database table. It cannot be null and must be unique.

Example:

`CustomerId` is the primary key of the `Customers` table.

---

15. What is a foreign key?

A foreign key is a column in one table that references the primary key of another table to establish a relationship and ensure referential integrity.

Example:

`CustomerId` in the `Orders` table points to the primary key `CustomerId` in the `Customers` table.

---

16. What is a one-to-many relationship?

It means one record in a table can be linked to multiple records in another table.

Example:

```
Customer (1) ---- (*) Orders
```

One customer can place many orders, but each order belongs to only one customer.

---

17. Why do we use JOIN?

We use a SQL `JOIN` to retrieve combined data from two or more related tables in a single query based on matching columns.

Example:

Joining the `Orders` table with the `Customers` table to display the customer's name alongside their order details.

---

18. What is the difference between table and entity?

An entity is a class in our application code representing a domain concept. A table is the physical database structure composed of rows and columns where that data is permanently stored.

Example:

`Customer.cs` is the C# entity class in our program, while `Customers` is the actual table inside SQL Server.

---

19. Why do we use GitHub?

We use GitHub to store code in the cloud, track changes (version control), safeguard our files, and easily collaborate with other developers on the same project.

---

20. What makes a README useful?

A good README acts as the front page of the project, explaining what the application does, how to set it up, how to run it, and its core architecture so others can immediately get started.

- Clear Project Description
- List of Technologies Used
- Detailed Installation & Execution Steps
- Project Directory Structure

---

21. Why are multiple commits better than one final commit?

Multiple logical commits show a clear evolutionary step-by-step history, make code reviews easier, and allow us to isolate bugs or roll back specific updates easily without losing all our work.

Example of Git history:

```
feat: add Employee and Customer models
feat: implement BankService business logic
test: write unit tests for account transfer logic
docs: document setup in README.md
```

---

22. Why is professional delivery important?

Professional delivery proves that you do not just write code, but can package it professionally. It shows attention to detail, adherence to standards, and respect for the team by providing clean, tested code and excellent documentation.